UNDB
CENTRO UNIVERSITÁRIO

COSEG Mobilidade

O servidor que impede conflitos na reserva de veículos do Porto do Itaqui

Programação para Web | **PBL 1** | **Python + Django**

Reservas por telefone e planilha geravam carros duplicados, lotados e horários sobrepostos

8 veículos leves até 4 passageiros

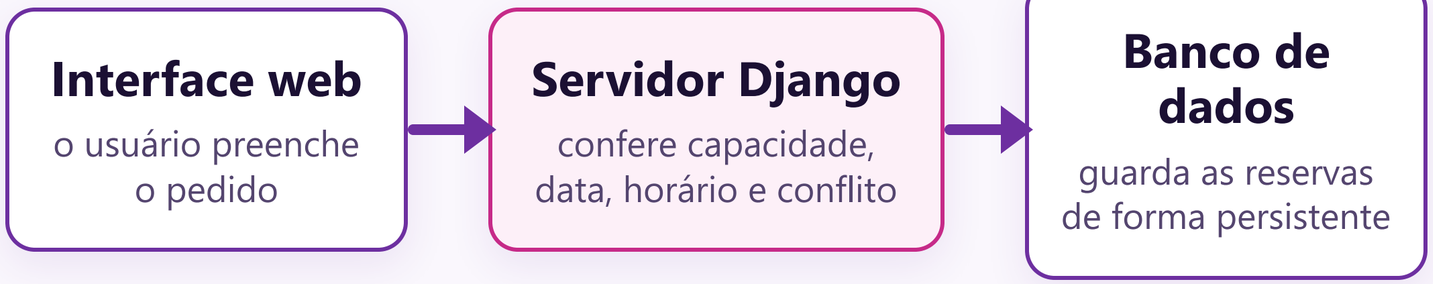
2 veículos coletivos até 18 passageiros

Pedidos chegavam sem padrão:

telefone · mensagem · e-mail · balcão · planilhas

1. Duas equipes esperaram o mesmo veículo para atividades diferentes.
2. Um carro de 4 lugares foi pedido para 7 pessoas e ninguém percebeu.
3. O horário de retorno era ignorado: o carro das 14h foi prometido de novo às 14h.

Um servidor central guarda todas as reservas e recusa o que for inválido



1. A interface web envia o pedido de reserva ao servidor.
2. O servidor confere as regras: capacidade, data, horário e conflito.
3. Só o que passa nas regras é gravado no banco de dados.

O projeto se divide em arquivos, e cada um tem uma função simples

`models.py`	Tabelas: Veículo e Reserva
`services.py`	Regras: período, conflito, disponibilidade
`views.py`	API: login, CRUD e cancelamento
`urls.py`	Endereços (rotas) do servidor
`forms.py`	Valida os dados recebidos
`seguranca.py`	Login obrigatório
`tests.py`	42 testes automáticos

1. models.py define o que o sistema guarda.
2. services.py reúne as regras de negócio.
3. views.py e urls.py recebem os pedidos e respondem.

O banco guarda duas coisas: a frota de veículos e as reservas feitas

models.py

```
29  class Veiculo(models.Model):
34      codigo = models.CharField(max_length=10, unique=True)
35      categoria = models.CharField(max_length=10, choices=Categoria.choices)
36      capacidade = models.PositiveSmallIntegerField(editable=False)
37      ativo = models.BooleanField(default=True)

72  class Reserva(models.Model):
77      solicitante = models.CharField(max_length=120)
78      setor = models.CharField(max_length=80)
82      data = models.DateField()
83      hora_saida = models.TimeField()
86      hora_retorno = models.TimeField()
87      passageiros = models.PositiveSmallIntegerField()
88      veiculo = models.ForeignKey(Veiculo, on_delete=models.PROTECT,
    related_name="reservas")
```

1. Veículo: código, categoria e capacidade (que vem da categoria).
2. Reserva: quem pediu, para onde, quando e quantas pessoas.
3. Cada reserva aponta para um veículo.

Antes de salvar, o servidor confere se todos os passageiros cabem no veículo

services.py

```
8  CAPACIDADES = {"LEVE": 4, "COLETIVO": 18}
```

models.py

```
200 if pax < 1:
201     add("passageiros", "passageiros_invalido", "Informe ao menos 1
    passageiro.")
202 elif pax > MAX_PASSAGEIROS:
203     add("passageiros", "acima_do_limite",
204         f"Solicitação rejeitada: o máximo é {MAX_PASSAGEIROS} passageiros.")
205 else:
206     if v is not None and pax > v.capacidade:
207         add("passageiros", "capacidade_excedida",
208             f"O veículo {v.codigo} comporta {v.capacidade} passageiros "
209             f"e foram solicitados {pax}.")
```

1. Leve leva até 4 pessoas. Coletivo, até 18.
2. Mais de 18 pessoas: o pedido é rejeitado.
3. 7 pessoas num carro de 4? Erro claro, antes da saída.

O servidor não aceita data no passado nem retorno antes da saída

services.py

```
32      if checar_passado and not permitir_data_passada():
33          agora = timezone.localtime()
34          if data < agora.date():
35              erros.append(("data", "data_passada", "A data não pode estar no passado."))
54      if data_retorno_ok and hora_saida is not None and hora_retorno is not None:
55          inicio, fim = intervalo(data, hora_saida, hora_retorno, data_retorno)
56          if fim <= inicio:
57              msg = "O horário de retorno deve ser posterior ao de saída."
10  MAX_DIAS_VIAGEM = 1  # o retorno pode ser, no máximo, 1 dia depois da saída
```

1. Data anterior a hoje: recusada.
2. O retorno precisa ser depois da saída.
3. Viagem após a meia-noite vale, com data de retorno (máx. 1 dia).

Há conflito quando o mesmo veículo é pedido em períodos que se sobrepõem



services.py

```
79 def reservas_conflitantes(veiculo, data, saida, retorno, data_retorno=None,
   ignorar_pk=None):
81     inicio, fim = intervalo(data, saida, retorno, data_retorno)
82     candidatas = reservas_ativas_no_periodo(data, data_retorno or data,
   ignorar_pk).filter(veiculo=veiculo)
83     return [r for r in candidatas if r.inicio < fim and r.fim > inicio]
```

1. Reserva existente: VL-01, das 8h às 10h.
2. Pedido das 9h às 11h se sobrepõe: é recusado.
3. Pedido das 10h30 às 12h não se sobrepõe: é aceito.

Se o usuário informa só a categoria, o servidor escolhe o menor veículo livre

services.py

```
96  ocupados = {
97      r.veiculo_id
98      for r in reservas_ativas_no_periodo(data, data_retorno or data,
    ignorar_pk)
99      if r.inicio < fim and r.fim > inicio
100 }
101 qs = Veiculo.objects.filter(ativo=True,
    capacidade__gte=passageiros).exclude(pk__in=ocupados)
102 if categoria:
103     qs = qs.filter(categoria=categoria)
104 return qs.order_by("capacidade", "codigo")  # sugere o menor veículo que
    atende
```

1. Primeiro, descarta os veículos já ocupados no período.
2. Depois, mantém só os ativos com lugares suficientes.
3. Ordena do menor para o maior e sugere o primeiro.

O servidor oferece endereços (rotas) para cadastrar, consultar e cancelar reservas

urls.py

```python
urlpatterns = [
    path("csrf/", views.csrf),
    path("login/", views.login_view),
    path("logout/", views.logout_view),
    path("eu/", views.eu),
    path("veiculos/", views.veiculos_lista),
    path("veiculos/<int:pk>/", views.veiculo_detalhe),
    path("reservas/", views.reservas_lista),
    path("reservas/<int:pk>/", views.reserva_detalhe),
    path("reservas/<int:pk>/cancelar/", views.reserva_cancelar),
    path("disponibilidade/", views.disponibilidade),
```

1. Cadastrar e consultar veículos e reservas (CRUD).
2. Cancelar libera o veículo sem apagar o histórico.
3. Disponibilidade mostra quais veículos estão livres.

Só quem fez login usa o sistema, e a frota só pode ser alterada pelo COSEG

views.py

```
179     if cache.get(chave, 0) >= MAX_TENTATIVAS:
180         return _erro("Muitas tentativas de login. Aguarde alguns minutos.",
    status=429)
182     user = authenticate(request, username=username, password=senha)
183     if user is None:
184         cache.set(chave, cache.get(chave, 0) + 1, BLOQUEIO_SEGUNDOS)
185         return _erro("Usuário ou senha inválidos.", status=401)
```

```
145 def _negar_se_nao_coseg(request):
146     if not request.user.is_staff:
147         return _erro("Acesso restrito ao COSEG.", status=403,
    codigo="sem_permissao")
```

```
151 def _negar_se_alheia(request, reserva):
152     if request.user.is_staff or reserva.criado_por_id == request.user.id:
```

1. Login obrigatório. 5 erros bloqueiam por 5 minutos.
2. Só o COSEG cria ou altera veículos.
3. Cada pessoa vê só as próprias reservas.

Cenários reais comprovam que o servidor aceita o válido e recusa o inválido

Pedido 10h30–12h (sem conflito) Reserva registrada com sucesso. Veículo: VL-01.	201
Atribuição automática por categoria Reserva registrada com sucesso. Veículo: VL-02.	201
7 pessoas em carro de 4 lugares O veículo VL-02 comporta 4 passageiros e foram solicitados 7.	400
19 pessoas (acima do limite) Solicitação rejeitada: o máximo é 18 passageiros.	400
Retorno antes da saída O horário de retorno deve ser posterior ao de saída.	400
Campos obrigatórios em branco Este campo é obrigatório.	400
Pedido 9h–11h (VL-01 já reservado) Conflito: VL-01 já está reservado nesse intervalo.	409

42 testes automáticos do projeto: todos passaram

1. Reservas válidas retornam 201 (criado).
2. Regras violadas retornam 400, com mensagem clara.
3. Conflito de horário retorna 409.

Resposta ao PBL: o servidor Django é a fonte de verdade e barra erros antes de gravar

Cadastro e persistência da frota	`models.py`
Estrutura de dados da reserva	`models.py`
Campos obrigatórios validados	`forms.py`
Capacidade do veículo validada	`models.py`
Data e horários validados	`services.py`
Conflitos reais, consultando o banco	`services.py`
Mensagens de erro e sucesso em JSON	`views.py`
Rotas organizadas para a interface	`urls.py`

1. Persiste os dados: banco, models e migrations.
2. Aplica as regras de capacidade, data, horário e conflito.
3. Rotas organizadas e respostas claras em JSON.